Recoup

Autonomous B2B Receivables & Promise-to-Pay Agent

Complete Backend Technical Documentation

Generated: 06 September 2026

Version 1.0 | Track 03: AI Revenue Recovery | Razorpay AI Buildathon 2026

Table of Contents

1. Executive Overview

India's Economic Survey (Budget 2026) put ■8.1 lakh crore currently stuck in delayed payments owed to MSMEs nationally. A 2026 industry report from Recordent found the average Indian SME is carrying ■3.83 crore in overdue receivables at any given time. This is one of the most pervasive working-capital problems a B2B business in India faces.

Recoup is an autonomous AI agent that decides who to chase for overdue B2B invoices, how hard, and when to stop — then proves exactly how much money it recovered. Built on FastAPI (Python 3.11+), PostgreSQL, Razorpay Payment Links, and a provider-agnostic LLM client, Recoup removes the judgment call of accounts-receivable follow-up from a person's plate without removing their control.

1.1 What Makes Recoup Different

Generic Dunning Script	Recoup
Same email to every overdue customer	Ranks by expected recovery value; low-risk invoices left alone
Reminds until someone stops it	Hard escalation ladder with automatic stop — no indefinite nagging
Any discount the LLM feels like offering	Policy-enforced discount ceiling the LLM cannot exceed
'Reminder sent' is the whole log	Every action carries a reason; full decision trail queryable
Success = emails sent	Success = batch-level ■ recovered, measured not claimed
No memory of what was promised	Promises recorded, watched against deadline, drive next action

2. High-Level Architecture

The diagram below shows the complete end-to-end data flow of Recoup, from the moment an overdue invoice enters the system through to the final batch evaluation report.

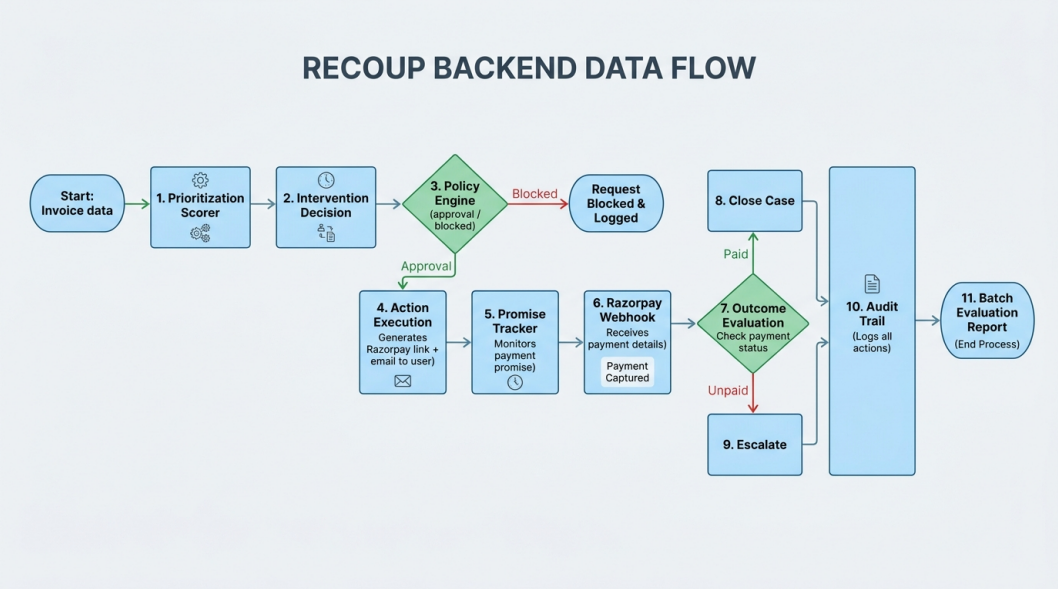

Figure 1 – Recoup Backend Data Flow

2.1 Core Design Principle: Score → Propose → GATE → Transition → Execute

Every action the agent takes passes through exactly five ordered steps. The order is the safety property, not a style choice. Execution receives a PolicyDecision, never a raw ProposedAction, so there is no code path that can send a message without a gate verdict attached.

Step	Module	Decides
score	app/core/scorer.py	What is this invoice worth acting on?
propose	app/core/agent.py	What is the next rung, and how hard do we push?
gate	app/core/policy.py	Is that action permitted by policy?
transition	app/core/escalation.py	May this case move to the next state?
execute	app/services/	Send it, and log that we did

> *Both approved AND blocked actions are recorded by app/core/audit.py. A blocked action leaves as much evidence as a sent one.*

2.2 Layering: Why the Core Has No Database

app/core/ operates on plain Pydantic snapshots (CaseSnapshot), not ORM rows. Two adapters feed it: one from the synthetic generator for batch demos, one from the database for live API operation. This achieves three goals:

1. The demo runs on a clean checkout with no Postgres, no API keys, no network.
2. Safety tests are unit-testable in milliseconds against in-memory objects.

3. One scorer, one gate, two data sources — the API and batch paths converge on the same CaseSnapshot.

3. Component Deep-Dive

3.1 Prioritization Scorer (app/core/scorer.py)

The scorer ranks every open invoice by expected recovery value:

```
EV = P(recovery) x outstanding x urgency_weight - intervention_cost
```

This mirrors credit-risk practice (Expected Loss = PD x EAD x LGD) run in reverse. The scorer returns one of three tiers: WAIT (negative EV), REMIND (gentle contact), ESCALATE (high value-at-risk).

The rules-based scorer declares fallback_used=True on every prediction. An optional trained model (XGBoost + Logistic Regression + MLP, isotonically calibrated) is available with USE_MODEL_SCORER=true.

Trained model vs rules-based (batch of 600, seed 42):
AUC: 0.709 vs 0.667 | Brier: 0.1944 vs 0.2433 | Top-120 value-at-risk: +■5.9L (+7.5%)

3.2 Policy / Gate Engine (app/core/policy.py)

The policy engine is the single choke-point every outbound action must clear. Built on venmo/business-rules (Wave 4 adds regopy, in-process OPA). Rules are data: GET /api/v1/policy serves the compiled rule set verbatim.

Two invariants:
• Blocks are absolute; adjustments only reduce. A rule can refuse or lower a ceiling, never approve or raise.
• Every violation is reported, not just the first. stop_on_first_trigger is off.

Parameter	Value
MAX_DISCOUNT_PERCENT	Hard ceiling on any discount offer (e.g. 10%)
MAX_CONTACT_FREQUENCY_DAYS	Minimum gap between contacts per invoice (e.g. 3 days)
ESCALATION_LADDER_DAYS	Day offsets for each rung, e.g. 1,3,7,14

3.3 Escalation State Machine (app/core/escalation.py)

Built on pytransitions/transitions with auto_transitions=False. The only way a case can move is via the four declared triggers.

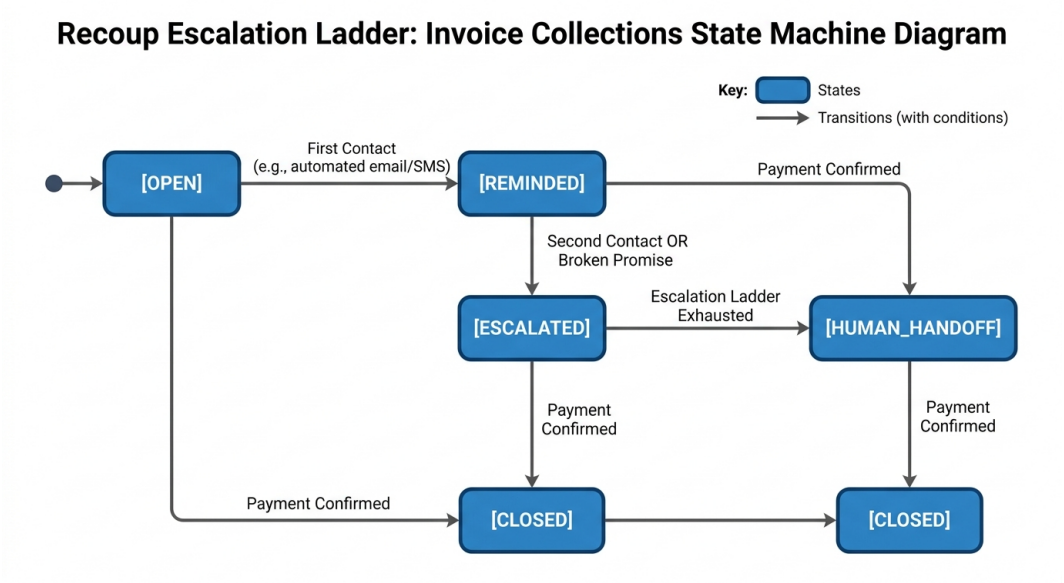

Figure 2 – Escalation State Machine

```
MONITORING   ->  send_reminder   ->  REMINDED
REMINDED     ->  escalate        ->  ESCALATED
ESCALATED    ->  handoff         ->  HUMAN_HANDOFF
Any state    ->  close           ->  CLOSED
```

Guards (contact_allowed, ladder_step_due) are conditions on transitions, not post-hoc checks. A blocked transition simply does not happen — the trigger returns False and state is unchanged.

3.4 Promise-to-Pay Tracker (app/core/promise_tracker.py)

A promise is evidence of intent, never evidence of payment. Rules:
• Only a signature-verified Razorpay webhook sets PAID. Never a promise alone.
• A broken promise escalates exactly one rung per the ladder.
• Superseded promises are kept, not deleted ('rescheduled three times' is a pattern reviewers need).
• An opt-out reply permanently blocks all future automated contacts for that invoice.

3.5 Reply Understanding (src/ml/reply/)

Two-stage cascade classifier:

Stage A: TF-IDF/SVM (joblib artifact) — handles 35.6% of replies at 0.917 accuracy. Grouped macro-F1: 0.764.

Stage B: LLM via instructor (Groq/Gemini/Anthropic) — handles ambiguous cases with validation-aware retry.

If both stages fail: classified as OTHER, routed to GET /api/v1/replies/review for human review. Never guesses.

Intent labels: PROMISE_TO_PAY, DISPUTE, OPT_OUT, ACKNOWLEDGEMENT, OTHER

3.6 Decision Trace / Audit Ledger (app/core/audit.py)

Append-only, hash-chained ledger. Each entry's entry_hash is SHA-256 over its own content plus its predecessor's hash. Any edit, reorder, or deletion breaks all subsequent hashes. DecisionLedger.verify() reports the first discrepancy.

Wave 4 adds event sourcing (eventsourcing library): every Decision Trace append is dual-written as a semantic domain event, reconstructible at any version via get_case(invoice_id, version=N).

3.7 Action Executor (app/services/)

Razorpay (razorpay_client.py): Creates test-mode Payment Links. DRY_RUN=true (default) logs without creating, consuming contact caps so a dry run behaves like live.

Resend (resend_client.py): Email delivery. Signed Reply-To encodes invoice_id for reply routing.

LLM (llm_client.py): Drafts explanation/reminder text ONLY. Cannot decide amounts or bypass the gate.

3.8 Webhook Receiver (app/api/webhooks.py)

The only writer of PAID status. Three ordered properties:
1. HMAC-SHA256 verification before any payload parsing.
2. Idempotency on event_id — duplicate returns 200 without re-processing.
3. Acknowledge what was stored: unparseable payload returns 400, not 500.

4. Database Schema

PostgreSQL via SQLAlchemy 2.0 (async, psycopg3). Schema owned by Alembic — the application never creates tables at startup.

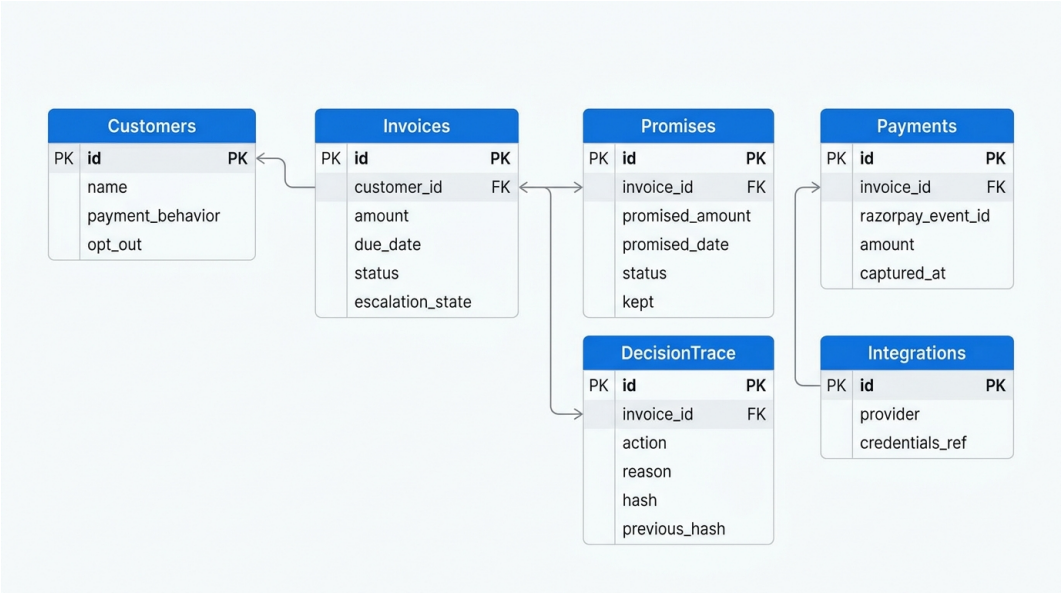

Figure 3 – Entity-Relationship Diagram

4.1 Tables

Table	Purpose
customers	Customer profiles: payment behaviour archetype, opt-out flag
invoices	Core invoice record: amount, due_date, status, escalation_state
promises	Promise-to-pay commitments: promised_amount, promised_date, kept flag
decision_trace	Immutable hash-chained audit log of every decision
payments	Webhook-confirmed Razorpay payment events
integrations	Credentials references for Razorpay, Resend, LLM
run_summaries	Aggregate metrics from each batch run cycle

4.2 Migration Strategy

All schema changes go through Alembic: alembic upgrade head runs before every deploy. The application performs a SELECT 1 connectivity check at startup and refuses to serve if the DB is unreachable.

5. The Agentic Decision Loop

The agent runs in batch mode (POST /api/v1/tasks/run-batch, advisory-locked) or per-invoice on demand (POST /api/v1/invoices/{id}/run-cycle). The decision loop is identical in both modes.

5.1 Step-by-Step Walkthrough

Given three invoices:

```
INV-1044  Rs 75,000   2 days overdue   Customer: strong payment history
INV-1042  Rs 50,000   5 days overdue   Customer: reliably pays late but pays
INV-1043  Rs 2,00,000 12 days overdue  Customer: high-value, new risk signals
```

Step 1 - Score:
INV-1044 P(recovery)=0.84 -> REMIND | INV-1042 -> REMIND | INV-1043 -> ESCALATE

Step 2 - Propose:
INV-1043: 'Pay Rs 1,80,000 by Friday; Rs 20,000 late fee waived' (Rs 20,000 = configured ceiling)

Step 3 - Gate:
All three pass contact-frequency cap. INV-1043 discount 10% <= 10% ceiling -> APPROVED.

Step 4 - Transition:
Each invoice advances one rung on the state machine.

Step 5 - Execute:
Razorpay Payment Link generated. Resend email dispatched. All appended to Decision Trace.

Step 6 - Promise Tracking:
Customer replies 'I'll pay Friday' -> recorded as promise.
Friday arrives, no webhook -> promise broken -> INV-1043 escalates one rung.

5.2 Batch Evaluation Results (600 invoices, 5 cycles, seed 42)

Parameter	Value
Invoices processed	600
Total overdue value	Rs 6.96 Cr
Cases flagged for intervention	574
Cases left alone	26
Interventions executed	580
Actions blocked by policy	699
Recovered (of flagged)	385 (67.1%)
Recovered value	Rs 4.68 Cr

False / unnecessary interventions	316
Compliance violations	0
Ledger hash chain verified	Yes
Decision trace entries	5,807

6. API Surface

Versioned REST API at /api/v1/. Interactive docs at /docs only when DEBUG=true (off by default).

Method	Endpoint	Purpose
GET	/api/v1/health	Liveness check
GET	/api/v1/health/db	Database connectivity check
POST	/api/v1/invoices/batch	Ingest a batch of invoices and customers
GET	/api/v1/invoices/{id}	View one invoice's current state
GET	/api/v1/invoices/{id}/audit	Full decision trail for one invoice
POST	/api/v1/invoices/{id}/run-cycle	Manually trigger one agent decision cycle
POST	/api/v1/webhooks/razorpay	Razorpay webhook receiver (HMAC-verified)
POST	/api/v1/replies	Inbound email reply from Resend webhook
GET	/api/v1/replies/review	Human review queue for ambiguous replies
GET	/api/v1/reports/batch	Batch evaluation report
GET	/api/v1/forecast/cash	Probabilistic 7/30-day cash forecast
GET	/api/v1/forecast/cash/card	Validation metrics for the cash forecast
GET	/api/v1/policy	Currently configured policy
POST	/api/v1/tasks/run-batch	Trigger autonomous batch run (advisory-locked)
GET	/api/v1/secrets/status	Secret provider health and last-refresh time
POST	/api/v1/secrets/refresh	Manually trigger secret refresh from provider

7. Technology Stack

Layer	Choice	Why
API framework	FastAPI + Uvicorn (Python 3.11+)	Typed contracts, async-native, Pydantic v2
Database	PostgreSQL / SQLAlchemy 2.0 async / psycopg3	Relational integrity; Alembic migrations
Local dev DB	SQLite via aiosqlite	Zero-infrastructure demo and CI
Payments	Razorpay Payment Links + Webhooks	Real verifiable lifecycle; zero financial risk
Email	Resend	Fast setup; signed Reply-To routing
LLM	Groq / Gemini / Anthropic (agnostic)	Zero-cost demo; reasoning never decides amounts
Policy engine	business-rules + regopy (in-process OPA)	Rules as data; Rego < 20ms p99
Escalation FSM	pytransitions/transitions	auto_transitions=False enforces ladder
Structured LLM	instructor	Schema-bound output with validation-aware retry
Event sourcing	eventsourcing	Aggregate stream; time-travel replay
ML training	XGBoost, scikit-learn, SHAP	Recovery scorer; calibrated; explainable
Rate limiting	Redis (hiredis) sliding window	Shared state across replicas
Observability	OpenTelemetry SDK + OTLP	Traces + metrics; console fallback locally
Logging	structlog + python-json-logger	Structured JSON logs; request-ID correlation
Hosting	Render (backend) + Vercel (frontend)	Single public URL; no infra overhead

8. External Service Integrations

8.1 Razorpay

app/services/razorpay_client.py wraps the razorpay SDK (>=2.0).
Payment Link creation: accepts amount, description, customer details, expiry. DRY_RUN=true logs the link body without creating it.
Webhook: HMAC-SHA256 before any parsing. Idempotent on event_id. Handles payment.captured (-> PAID) and payment_link.expired (-> re-score).

8.2 Resend Email

app/services/resend_client.py:
Sends HTML/text emails. Signed Reply-To encodes invoice_id (HMAC): reply+{invoice_id}@{RESEND_DOMAIN}. Inbound replies arrive at POST /api/v1/replies and are routed without any DB lookup.

8.3 LLM Client (Provider-Agnostic)

Selects provider from LLM_PROVIDER env var (groq | gemini | anthropic).
Two function types:
generate() -> free-text: reminder copy, escalation narration.
generate_structured() -> instructor-wrapped schema-bound calls (reply classification, promise extraction).

8.4 Secrets Management

Priority order: Render env -> Doppler -> AWS Secrets Manager.
inject_secrets_into_env() runs first in lifespan startup, before any other service. Cached with TTL (SECRET_CACHE_TTL_HOURS, default 1h). Background refresh every SECRET_REFRESH_INTERVAL_SECONDS (default 300s).
Secret values are scrubbed before any structlog emission.

9. Machine Learning Models

9.1 Recovery Probability Model (src/ml/recovery/)

Three candidates trained on the same temporal split: Logistic Regression (baseline), XGBoost, small MLP. Each isotonically calibrated on a held-out validation split.

Metric	Result
AUC	0.709 (trained) vs 0.667 (rules)
Brier score	0.1944 (trained) vs 0.2433 (rules)
ECE	0.065 (trained) vs 0.180 (rules)
Top-120 at risk	Rs 85.5L (trained) vs Rs 79.5L (rules) (+7.5%)

Top SHAP features:

Feature	Mean |SHAP|
recency_weighted_on_time_score	0.469
customer_on_time_ratio_all_time	0.275
customer_invoice_count	0.271
prior_promise_kept	0.218
customer_avg_days_late	0.214
current_escalation_tier	0.153

9.2 Reply Intent Classifier (src/ml/reply/)

Two-stage cascade:
Stage A: TF-IDF + Linear SVM (sklearn Pipeline). Handles 35.6% of replies at 0.917 accuracy. Grouped macro-F1: 0.764.

Stage B: LLM via instructor for ambiguous cases. Retries with the specific validation error.

Intent labels: PROMISE_TO_PAY, DISPUTE, OPT_OUT, ACKNOWLEDGEMENT, OTHER.

9.3 Cash Forecast (src/ml/cash_forecast/)

Monte Carlo simulation over the at-risk book. Samples from calibrated P(recovery) distributions to produce P10/P50/P90 cash-in forecasts over 7-day and 30-day horizons. Validation card at GET /api/v1/forecast/cash/card.

9.4 Contact Timing (src/ml/contact_timing/)

Segmented Thompson Sampling bandit. Customers segmented by payment-behaviour archetype. Within each segment the bandit learns which day/time contact slots yield highest promise-to-pay conversion, updating Beta posteriors on each observed outcome.

9.5 Drift Detection (src/ml/drift/)

Detects customers whose payment behaviour is shifting from their historical archetype. Classifier trained on rolling payment ratio, days-late trend, dispute velocity. A drifting customer receives a higher urgency weight in the scorer.

10. Security Model

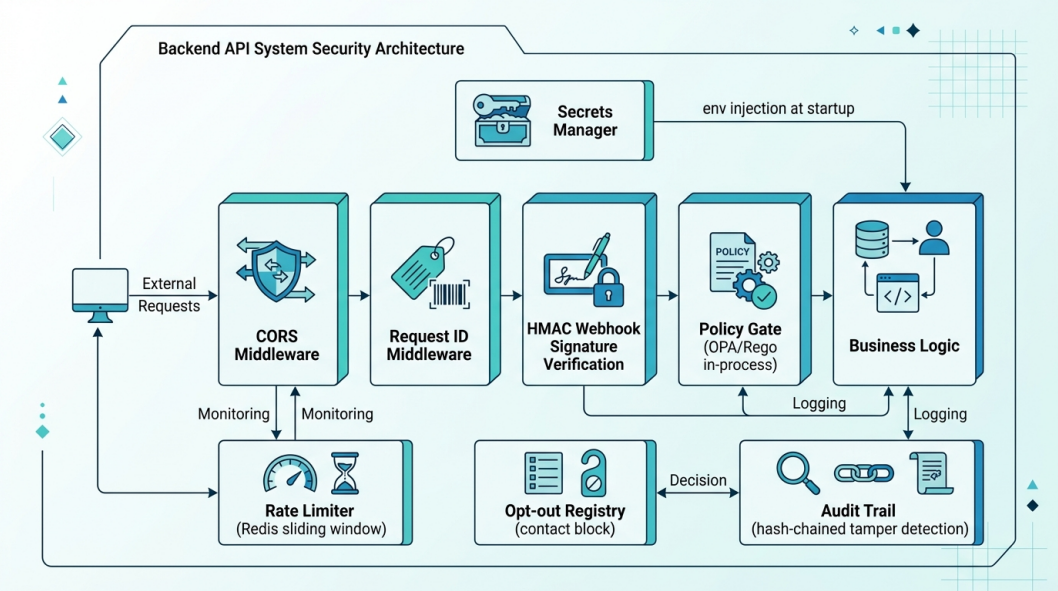

Figure 4 – Layered Security Architecture

10.1 Request Layer

CORS Middleware: explicit origin list (never wildcard). Allow-credentials only when specific origins named.

Request ID Middleware: every request receives a UUID correlation ID injected into structlog context and propagated through all trace spans.

10.2 Webhook Security

HMAC-SHA256 verified against RAZORPAY_WEBHOOK_SECRET before any payload parsing. Invalid signature returns 400 immediately. Verification uses the raw request body to prevent encoding normalisation from invalidating the signature.

10.3 Policy Gate as Security Boundary

The gate is not just a business-rule checker — it is a hard security boundary:
• Discount ceiling: LLM cannot offer more than MAX_DISCOUNT_PERCENT regardless of text generated.
• Opt-out registry: once set, blocks all automated contacts permanently. No bypass.
• Contact frequency cap: prevents customer harassment and regulatory exposure.

Wave 4 replaced business-rules with regopy (rego-cpp). Config values are a per-tenant data document the request cannot modify.

10.4 Audit Trail Integrity

Hash-chained append-only ledger. Any edit, reorder, or deletion breaks every subsequent hash. Verified in the batch evaluation report and in unit tests.

10.5 Secrets Security

Control	Implementation
No secret logging	Values scrubbed before any structlog emission
Provider auth	IAM roles / service accounts where supported
Rotation validation	POST /api/v1/secrets/refresh + GET /api/v1/secrets/status
PII in traces	Span attributes sanitised; customer names/emails excluded
OTLP auth	Configurable auth headers on OTLP exporter endpoint

11. Observability

11.1 OpenTelemetry Tracing

Auto-instrumented: FastAPI (request spans), SQLAlchemy (query spans), httpx (outbound HTTP).

Manual span hierarchy:
batch.cycle -> batch.scoring_phase -> invoice.score (per invoice)
batch.cycle -> batch.decision_phase -> invoice.decision
batch.cycle -> batch.execution_phase -> invoice.execute

Export: OTLP to configured endpoint. Falls back to ConsoleSpanExporter (stdout JSON) when OTEL_EXPORTER_OTLP_ENDPOINT is not set.

11.2 Structured Logging

structlog emits JSON logs with: request_id, invoice_id, customer_id, action, level, timestamp, module, function.
LOG_LEVEL env var controls verbosity (default INFO in production, DEBUG locally).

11.3 Health Endpoints

Endpoint	Checks
GET /api/v1/health	Application liveness
GET /api/v1/health/db	Database connectivity (SELECT 1)
GET /api/v1/ready/enhanced	DB + secret provider + OTLP exporter health

11.4 Key SLOs

Metric	Target
Batch cycle p95 latency	< 30 seconds
Health check response time	< 500 ms
Secret refresh time	< 100 ms
Policy gate decision	< 20 ms p99 (in-process Rego)
Webhook verification + ack	< 200 ms

12. Scaling Considerations

12.1 Stateless API Design

All state lives in PostgreSQL. Horizontal scaling is a config change (increase Render instance count). The batch advisory lock (pg_try_advisory_xact_lock) prevents concurrent batch cycles across replicas without any additional coordination.

12.2 Rate Limiter

Redis sliding-window (redis[hiredis]>=5.0) when RATE_LIMIT_REDIS_URL is set. Falls back to in-process deque for a single replica. Adding Redis is the only change needed to make the frequency cap correct across replicas.

12.3 Database Connection Pooling

DB_POOL_MODE controls SQLAlchemy pool mode:
session — default; one connection per request.
transaction — connection returned to pool between transactions (PgBouncer-compatible).

12.4 LLM Concurrency

LLM calls are async (httpx-backed). The batch loop runs invoice decisions sequentially within a cycle to preserve causal ordering in the Decision Trace. Scoring (CPU-local, no writes) is safe to parallelise with asyncio.gather.

12.5 Multi-Tenant Roadmap

1. OIDC/JWT authentication (Auth0, Clerk).
2. Per-business row-level data isolation in Postgres (RLS or schema-per-tenant).
3. Per-tenant PolicyConfig document in the Rego data store.
4. Per-tenant ESCALATION_LADDER_DAYS, MAX_DISCOUNT_PERCENT, etc.

The policy gate is already designed for this: Wave 4's Rego evaluation takes a business_id-keyed data document, so per-tenant configuration is a data change, not a code change.

13. Deployment

13.1 Infrastructure

Component	Hosting
Backend API	Render free web service (Python, uvicorn)
Frontend dashboard	Vercel (Next.js)
Database	Neon or Supabase free tier (PostgreSQL)
Email inbound worker	Cloudflare Email Worker
Redis (optional)	Render Redis or Upstash

13.2 Release Process

```
1. alembic upgrade head    <- migrate schema before new code serves
2. uvicorn app.main:app    <- start application
3. Health check at /api/v1/health/db confirms DB reachable
4. install_cascade_classifier() binds the reply classifier
```

13.3 Environment Variables

Variable	Purpose
DATABASE_URL	Postgres connection string
RAZORPAY_KEY_ID	Razorpay test-mode key ID
RAZORPAY_KEY_SECRET	Razorpay test-mode key secret
RAZORPAY_WEBHOOK_SECRET	HMAC secret for webhook signature verification
RESEND_API_KEY	Resend email delivery API key
LLM_PROVIDER	groq | gemini | anthropic
GROQ_API_KEY / GEMINI_API_KEY	Set only the one matching LLM_PROVIDER
MAX_DISCOUNT_PERCENT	Hard ceiling on any offer
MAX_CONTACT_FREQUENCY_DAYS	Minimum gap between contacts per invoice
ESCALATION_LADDER_DAYS	Day offsets, e.g. 1,3,7,14
DRY_RUN	true (default) = log actions, don't send
USE_MODEL_SCORER	true = XGBoost scorer; false (default) = rules
APP_ENV	development | demo | production
DEBUG	true = /docs enabled; false = disabled
RATE_LIMIT_REDIS_URL	Redis URL for distributed rate limiting
OTEL_EXPORTER_OTLP_ENDPOINT	OTLP export target (console fallback if absent)

14. Project Structure

```
app/
  main.py                FastAPI app entry, lifespan, middleware
  api/                   Route handlers (15 routers)
  core/
    domain.py            CaseSnapshot read model
    scorer.py            Expected-value prioritisation
    policy.py            Gate (business-rules + OPA/Rego)
    escalation.py        State machine (pytransitions)
    promise_tracker.py   Promise-to-pay logic
    audit.py             Hash-chained Decision Trace
    agent.py             Decision cycle, end to end
    evaluation.py        Batch report computation
    eventsourcing/       Event store integration
  models/                SQLAlchemy ORM tables
  schemas/               Pydantic request/response contracts
  db/session.py          Async engine + session factory
  services/
    llm_client.py        Provider-agnostic LLM wrapper
    razorpay_client.py   Payment Links + webhook verification
    resend_client.py     Email delivery
    repository.py        All DB access (single module)
    batch_runner.py      Batch cycle orchestration

src/
  data/synthetic_generator.py   Seeded invoice+customer batch
  ml/
    recovery/            Recovery probability model
    reply/               Reply intent cascade classifier
    cash_forecast/       Monte Carlo cash forecast
    contact_timing/      Thompson Sampling bandit
    drift/               Payment behaviour drift classifier

alembic/                 Schema migrations
scripts/                 Batch demo, training, seeding, this script
tests/                   384 tests (no DB or network required)
frontend/                Next.js dashboard (Vercel)
cloudflare-email-worker/ Inbound email routing to API
docs/                    Architecture, model cards, evaluation report
```

15. Roadmap to Production

The current MVP is production-shaped but not production-hardened:

Item	Detail
Multi-tenant auth	OIDC/JWT + per-business row-level data isolation
WhatsApp delivery	Meta Business verification; email stands in for demo
Webhook idempotency	Harden double-counting protection for production payment infra
Online model retraining	Re-train recovery scorer from webhook-confirmed outcomes
Policy Simulation Engine	POST /policy/simulate: replay last quarter under proposed policy change
SetFit reply classifier	Fine-tune from 8-16 labelled examples once real reply data exists
Observability stack	Self-hosted Grafana + Tempo or Honeycomb for trace storage
Secrets + backup drills	Automated backup drill, rotation runbooks, alerting
Compliance review	Legal review of escalation cadence before any live customer contact
Kubernetes / Terraform	Infrastructure-as-code for multi-region production deploy

Appendix A — How to Read the Evaluation Metrics

Regenerate with:
python scripts/run_batch_demo.py --batch-size 600 --seed 42 --cycles 5 --use-model

Read all numbers with these four caveats, which the report itself repeats:

A.1 The agent did not cause these recoveries

The synthetic generator samples each invoice's outcome independently of what the agent does. The recovery rate is a targeting measure — did the agent act on the invoices that were going to be paid? — not a causal one. Measuring uplift requires a holdout arm this simulation does not have.

A.2 The scorer is rules-based by default

Probabilities from the rules-based scorer are not calibrated. Add --use-model to score with the trained, calibrated model.

A.3 False interventions are counted as a cost

A contacted customer whose invoice would have been recovered anyway appears in the false interventions row — not quietly dropped. That number going up is a real regression.

A.4 Compliance violations are counted from the ledger, not from intent

The detector walks the Decision Trace in sequence. A bug in the orchestrator cannot suppress the violation count because the count reads the audit record, not the orchestrator's own return values.